

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Load the dataset (using your local path)
df = pd.read_csv('../data/CaliforniaHousing.csv')

# First 5 records
display(df.head())

```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	La
0	5.3668	27.0	6.702899	1.028986	1793.0	3.248188	33.
1	3.4919	44.0	5.138418	1.031073	1137.0	3.211864	37.
2	1.6196	38.0	3.830116	1.077220	732.0	2.826255	37.
3	4.0926	42.0	4.525114	0.981735	717.0	3.273973	33.
4	3.9063	41.0	4.633540	0.900621	387.0	2.403727	37.

```

# Last 5 records
display(df.tail())

```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	L
95	4.1050	16.0	6.035963	1.041763	2515.0	2.917633	32.
96	6.6073	17.0	6.771499	0.939803	2734.0	3.358722	34.
97	5.0758	35.0	6.378000	1.054000	1727.0	3.454000	34.
98	5.5061	17.0	6.923129	1.134862	4956.0	3.341875	34.
99	2.5313	30.0	5.039384	1.193493	NaN	2.679795	33.

```

# Shape of the dataset
print(f"Dataset Shape: {df.shape}")

```

Dataset Shape: (100, 9)

```

# Column names
print(f"Column Names:\n{df.columns.tolist()}")

```

Column Names:

```

['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup',
'Latitude', 'Longitude', 'MedHouseValue']

```

```

num_samples = df.shape[0]
num_features = df.shape[1] - 1 # Assuming 1 target variable

```

```
print(f"Number of samples: {num_samples}")
print(f"Number of features: {num_features} (excluding target)")
```

Number of samples: 100
Number of features: 8 (excluding target)

```
num_samples = df.shape[0]
num_features = df.shape[1] - 1 # Assuming 1 target variable
```

```
print(f"Number of samples: {num_samples}")
print(f"Number of features: {num_features} (excluding target)")
```

Number of samples: 100
Number of features: 8 (excluding target)

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   MedInc                 100 non-null   float64
1   HouseAge               100 non-null   float64
2   AveRooms               99 non-null    float64
3   AveBedrms              100 non-null   float64
4   Population             97 non-null    float64
5   AveOccup               99 non-null    float64
6   Latitude               99 non-null    float64
7   Longitude              96 non-null    float64
8   MedHouseValue          100 non-null   float64
dtypes: float64(9)
memory usage: 7.2 KB
```

```
# Display statistical summary
df.describe()
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveO
count	100.000000	100.000000	99.000000	100.000000	97.000000	99.00
mean	3.658175	28.860000	5.102687	1.048504	1526.123711	3.036
std	1.441273	11.658456	1.155867	0.117671	1055.858865	0.648
min	1.225400	4.000000	2.096692	0.877301	240.000000	1.360
25%	2.626625	18.000000	4.236945	1.006378	785.000000	2.633
50%	3.491900	29.000000	5.152500	1.030534	1263.000000	2.935
75%	4.406250	37.000000	5.912333	1.077766	1959.000000	3.350
max	8.113200	52.000000	8.970968	1.990323	5587.000000	4.856

```
# Check for missing values
print("Missing values per column:")
print(df.isnull().sum())
```

```
# Check for duplicate records
num_duplicates = df.duplicated().sum()
print(f"\nNumber of duplicate records: {num_duplicates}")

# Remove duplicates if any exist
if num_duplicates > 0:
    df = df.drop_duplicates()
    print(f"Updated Dataset Shape after removing duplicates: {df.shape}")
```

Missing values per column:

MedInc	0
HouseAge	0
AveRooms	1
AveBedrms	0
Population	3
AveOccup	1
Latitude	1
Longitude	4
MedHouseValue	0

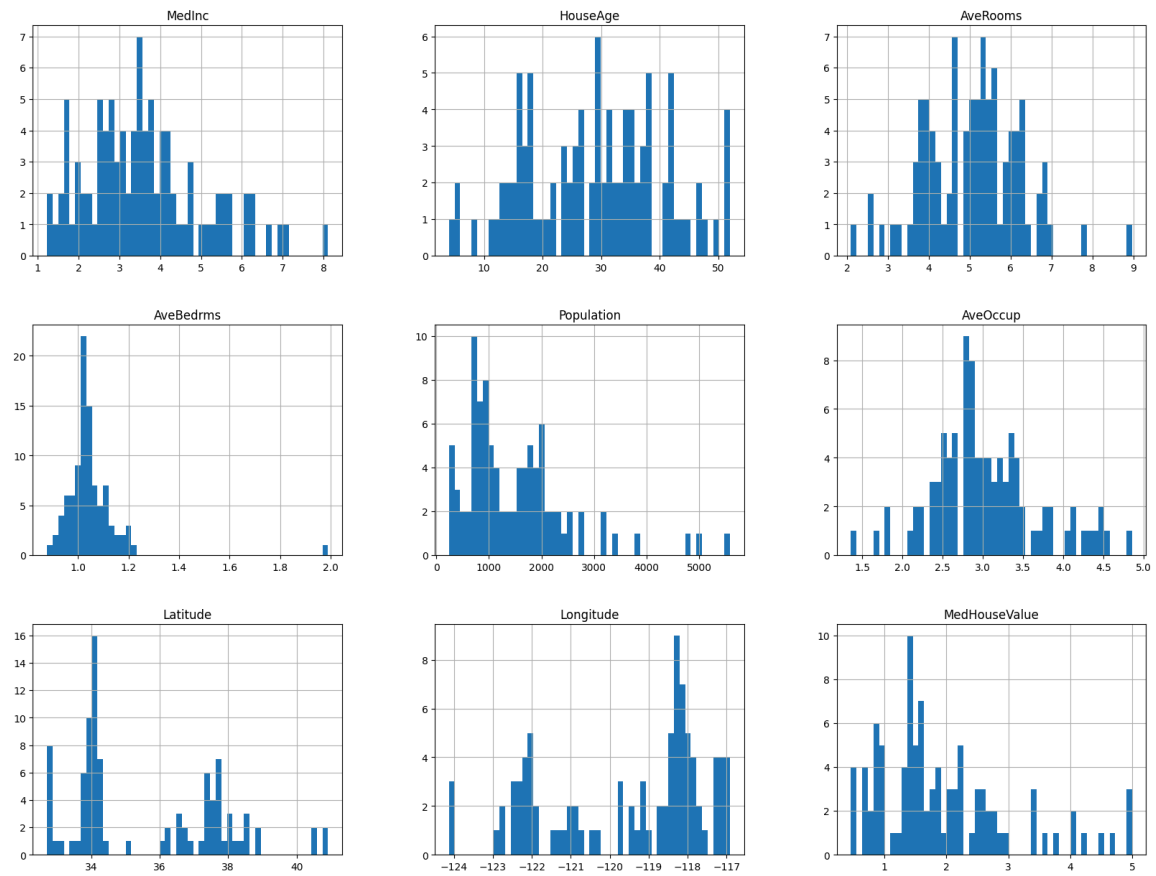
dtype: int64

Number of duplicate records: 4

Updated Dataset Shape after removing duplicates: (96, 9)

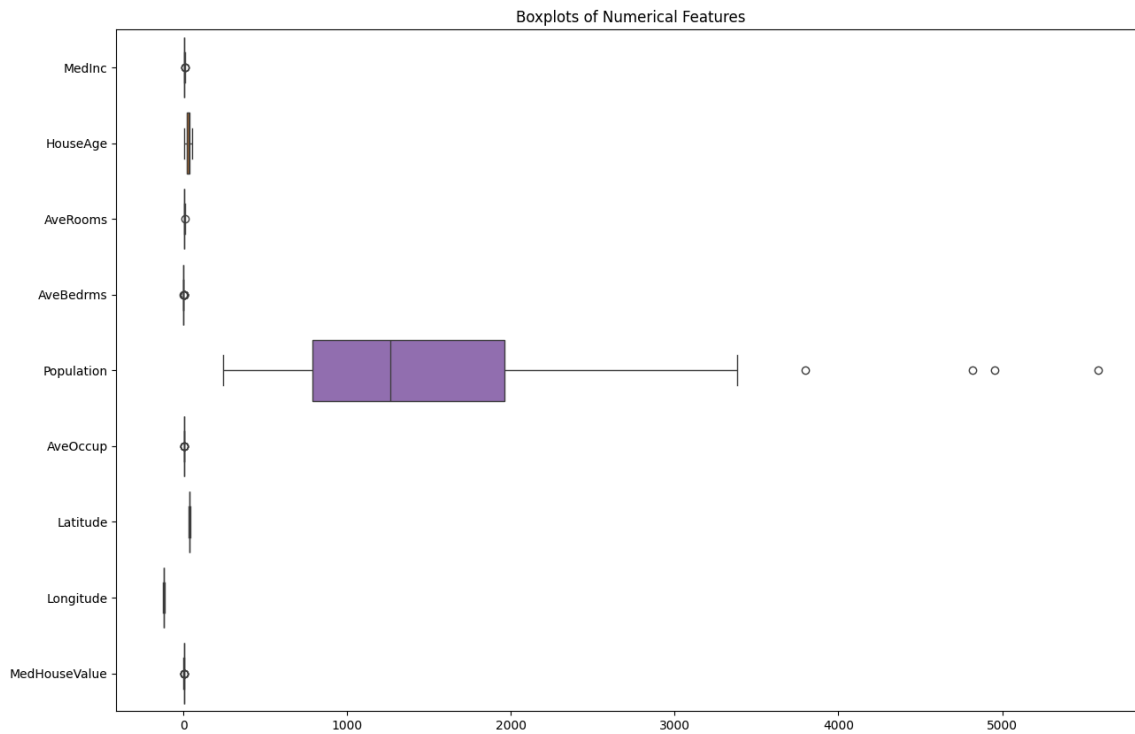
```
# Generate histograms for all numerical features
df.hist(bins=50, figsize=(20, 15))
plt.suptitle('Histograms of Numerical Features', fontsize=16)
plt.show()
```

Histograms of Numerical Features



Generate boxplots for numerical features

```
plt.figure(figsize=(15, 10))
sns.boxplot(data=df.select_dtypes(include=[np.number]), orient="h")
plt.title('Boxplots of Numerical Features')
plt.show()
```



Scatter plot

```
plt.figure(figsize=(10, 6))
sns.scatterplot(x='median_income', y='median_house_value', data=df,
                alpha=0.1)
plt.title('Median Income vs Median House Value')
plt.xlabel('Median Income (tens of thousands of $)')
plt.ylabel('Median House Value ($)')
plt.show()
```

ValueError Traceback (most recent call last)

Cell In[12], line 3

```
1 # Scatter plot
2 plt.figure(figsize=(10, 6))
----> 3 sns.scatterplot(x='median_income', y='median_house_value', data=df,
alpha=0.1)
4 plt.title('Median Income vs Median House Value')
5 plt.xlabel('Median Income (tens of thousands of $)')
6 plt.ylabel('Median House Value ($)')
```

File c:\Users\WAZNI\AppData\Local\Programs\Python\Python314\Lib\site-packages\seaborn\relational.py:615, in scatterplot(data, x, y, hue, size, style, palette, hue_order, hue_norm, sizes, size_order, size_norm, markers, style_order, legend, ax, **kwargs)

```
606 def scatterplot(
607     data=None, *,
608     x=None, y=None, hue=None, size=None, style=None,
(...) 612     **kwargs
613 ):
--> 615     p = _ScatterPlotter(
616         data=data,
617         variables=dict(x=x, y=y, hue=hue, size=size, style=style),
```

```

618         legend=legend
619     )
621     p.map_hue(palette=palette, order=hue_order, norm=hue_norm)
622     p.map_size(sizes=sizes, order=size_order, norm=size_norm)

```

File c:\Users\WAZNI\AppData\Local\Programs\Python\Python314\Lib\site-packages\seaborn\relational.py:396, in _ScatterPlotter.__init__(self, data, variables, legend)

```

387 def __init__(self, *, data=None, variables={}, legend=None):
388
389     # TODO this is messy, we want the mapping to be agnostic about
390     # the kind of plot to draw, but for the time being we need to set
391     # this information so the SizeMapping can use it
392     self._default_size_range = (
393         np.r_[.5, 2] * np.square(mpl.rcParams["lines.markersize"]))
394     )
--> 396     super().__init__(data=data, variables=variables)
398     self.legend = legend

```

File c:\Users\WAZNI\AppData\Local\Programs\Python\Python314\Lib\site-packages\seaborn_base.py:634, in VectorPlotter.__init__(self, data, variables)

```

629 # var_ordered is relevant only for categorical axis variables, and
may
630 # be better handled by an internal axis information object that
tracks
631 # such information and is set up by the scale_* methods. The
analogous
632 # information for numeric axes would be information about log scales.
633 self._var_ordered = {"x": False, "y": False} # alt., used
DefaultDict
--> 634 self.assign_variables(data, variables)
636 # TODO Lots of tests assume that these are called to initialize the
637 # mappings to default values on class initialization. I'd prefer to
638 # move away from that and only have a mapping when explicitly called.
639 for var in ["hue", "size", "style"]:

```

File c:\Users\WAZNI\AppData\Local\Programs\Python\Python314\Lib\site-packages\seaborn_base.py:679, in VectorPlotter.assign_variables(self, data, variables)

```

674 else:
675     # When dealing with long-form input, use the newer PlotData
676     # object (internal but introduced for the objects interface)
677     # to centralize / standardize data consumption logic.
678     self.input_format = "long"
--> 679     plot_data = PlotData(data, variables)
680     frame = plot_data.frame
681     names = plot_data.names

```

File c:\Users\WAZNI\AppData\Local\Programs\Python\Python314\Lib\site-packages\seaborn_core\data.py:58, in PlotData.__init__(self, data, variables)

```

51 def __init__(
52     self,

```

```

53     data: DataSource,
54     variables: dict[str, VariableSpec],
55 ):
56     data = handle_data_source(data)
--> 58     frame, names, ids = self._assign_variables(data, variables)
60     self.frame = frame
61     self.names = names

```

File c:\Users\WAZNI\AppData\Local\Programs\Python\Python314\Lib\site-packages\seaborn_core\data.py:232, in PlotData._assign_variables(self, data, variables)

```

230     else:
231         err += "An entry with this name does not appear in `data`."
--> 232     raise ValueError(err)
234 else:
235
236     # Otherwise, assume the value somehow represents data
237
238     # Ignore empty data structures
239     if isinstance(val, Sized) and len(val) == 0:

```

ValueError: Could not interpret value `median\_income` for `x`. An entry with this name does not appear in `data`.

<Figure size 1000x600 with 0 Axes>